

Java 21 features

Here is the list of Java 21 New Features:

- String Templates (Preview) [JEP-430]
- Sequenced Collections [JEP-431]
- Record Patterns [JEP-440]
- Pattern Matching for switch [JEP-441]
- Unnamed Patterns and Variables (Preview) [JEP-443]
- Unnamed Classes and Instance Main Methods (Preview) [JEP-445]
- Scoped Values (Preview) [JEP-446]
- Generational ZGC [JEP-439]
- Virtual Threads [JEP-444]
- Foreign Function & Memory API (Third Preview) [JEP-442]
- Vector API (Sixth Incubator) [JEP-448]
- Deprecate the Windows 32-bit x86 Port for Removal [JEP-449]
- Prepare to Disallow the Dynamic Loading of Agents [JEP-451]
- Key Encapsulation Mechanism API [JEP-452]
- Structured Concurrency (Preview) [JEP-453]

String Templates (Preview) [JEP-430]

String templates were previewed in Java 21 (JEP 430) and re-previewed in Java 22 (JEP 459). After some experience with the prototype, it became clear that the processor-centric nature of the proposed design was confusing to users and lacked the desired compositionality. String templates are removed in Java 23 and will not be a feature of Java 23, even as a preview feature. Here is the [update on String Templates](#) in JEP-459.

This feature builds on Text Blocks (introduced in Java 13) by providing placeholders for expressions in strings. It offers the insertion of values in a straightforward and type-safe manner.

To have an idea about this feature, we will just explore a simple example as shown below:

```
public class StringTemplatesExample {

    public static void main(String[] args) {

        String name = "Alice";

        int age = 30;

        String message = STR."Hello, ${name}. You are ${age} years old.";

        System.out.println(message);

    }

}
```

Sequenced Collections [JEP-431]

Sequenced Collections are a feature introduced in Java 21 to provide a unified interface for working with collections. This new feature provides simple methods to fetch the first and the last elements of a collection. The three new interfaces are:

- SequencedCollection
- SequencedSet
- SequencedMap

SequencedCollection

The *SequencedCollection* interface introduces several key methods:

1. **Element Access:**
 - `getFirst()`: Retrieves the first element.
 - `getLast()`: Retrieves the last element.
2. **Element Addition:**
 - `addFirst(E e)`: Adds an element to the front.
 - `addLast(E e)`: Adds an element to the back.
3. **Element Removal:**
 - `removeFirst()`: Removes and returns the first element.
 - `removeLast()`: Removes and returns the last element.
4. **Element Reversal:**
 - `reversed()`: Reverses the element of the Collection.

All are default methods except `reversed()`.

SequencedCollection Example

```

import java.util.ArrayList;
import java.util.SequencedCollection;

public class SequencedCollectionExample {

    public static void main(String[] args) {

        SequencedCollection<String> names = new ArrayList<>();

        // Adding elements
        names.addFirst("Alice");
        names.addLast("Bob");
        names.addFirst("Charlie");
        names.addLast("Robert");

        // Accessing elements
        System.out.println("First: " + names.getFirst());
        System.out.println("Last: " + names.getLast());

        // Removing elements
        names.removeFirst();
        System.out.println("After removeFirst: " + names);

        names.removeLast();
        System.out.println("After removeLast: " + names);
    }
}

```

Output

First: Charlie

Last: Robert

After removeFirst: [Alice, Bob, Robert]

After removeLast: [Alice, Bob]

SequencedSet

Below screenshot represents the declaration of SequencedSet. It extends the SequencedCollection & Set interfaces. It means SequencedSet inherits all the methods of SequencedCollection.

SequencedSet Example

```

import java.util.LinkedHashSet;
import java.util.SequencedCollection;

public class SequencedSetExample {

    public static void main(String[] args) {

        SequencedCollection<String> set = new LinkedHashSet<>();

        // Adding elements
        set.addFirst("Alice");
        set.addLast("Bob");
        set.addFirst("Charlie");

        // Accessing elements
        System.out.println("First: " + set.getFirst());
        System.out.println("Last: " + set.getLast());

        // Removing elements
        set.removeFirst();
        set.removeLast();

        System.out.println("Remaining: " + set);
    }
}

```

Output

First: Charlie

Last: Bob

Remaining: [Alice]

SequencedMap

The *SequencedMap* is useful for the *Map* interfaces & classes. It does not extend the *SequencedCollection*, but provides its own methods that apply the access order to map entries in place of individual elements. Here are some important methods of *SequencedMap*:

Modification Methods:

1. `putFirst(K key, V value)`
 - Adds or moves the specified key-value pair to the beginning of the map.
 - If the key already exists, its position is updated to the first.
2. `putLast(K key, V value)`
 - Adds or moves the specified key-value pair to the end of the map.
 - If the key already exists, its position is updated to the last.

Removal & Retrieval Methods:

1. `pollFirstEntry()`
 - Removes and returns the first key-value pair in the map.
 - Returns null if the map is empty.
2. `pollLastEntry()`
 - Removes and returns the last key-value pair in the map.
 - Returns null if the map is empty.

Access Methods:

1. `firstEntry()`
 - Retrieves (but does not remove) the first key-value pair in the map.
 - Returns null if the map is empty.
2. `lastEntry()`
 - Retrieves (but does not remove) the last key-value pair in the map.
 - Returns null if the map is empty.

SequencedMap Example

```
import java.util.LinkedHashMap;
import java.util.SequencedMap;

public class SequencedMapExample {

    public static void main(String[] args) {

        // Create a SequencedMap using LinkedHashMap
        SequencedMap<String, Integer> map = new LinkedHashMap<>();

        // Adding entries
        map.putFirst("Alice", 25);
        map.putLast("Bob", 30);
        map.putLast("Charlie", 35);

        // Accessing first and last entries
        System.out.println("First Entry: " + map.firstEntry()); // Output: Alice=25
        System.out.println("Last Entry: " + map.lastEntry()); // Output: Charlie=35

        // Removing entries
        map.pollFirstEntry();
        System.out.println("After removeFirst: " + map); // Output: {Bob=30, Charlie=35}
        map.pollLastEntry();
        System.out.println("After removeLast: " + map); // Output: {Bob=30}
    }
}
```

Output

First Entry: Alice=25

Last Entry: Charlie=35

After removeFirst: {Bob=30, Charlie=35}

After removeLast: {Bob=30}

Record Patterns [JEP-440]

Record Patterns (introduced in Java 21) extend pattern matching capabilities to [records](#), allowing destructuring and deconstruction of record components directly in switch statements, if conditions, and instanceof checks. This feature simplifies working with immutable data structures like records by enabling concise, readable, and declarative code. Below are the Key Features of Record Patterns.

1. Deconstruction of Record Components:

- Access the components of a record directly while pattern matching.
- Eliminates the need for boilerplate code to extract values.

2. Nested Patterns:

- Supports patterns within patterns, enabling deep matching of complex structures.

3. Integration with switch and instanceof:

- Allows using patterns for conditional logic in switch and instanceof.

Example: Basic Record Pattern

Java 21 Implementation: Using Record Patterns

```
public record Person(String name, int age) {}
```

```
public class RecordPatternExample {  
    public static void main(String[] args) {  
        Object obj = new Person("Alice", 30);  
  
        if (obj instanceof Person(String name, int age)) {  
            System.out.println("Name: " + name + ", Age: " + age);  
        }  
    }  
}
```

Output:

Name: Alice, Age: 30

Example: Nested Record Patterns

Java 21 Implementation: Nested Patterns

```
public record Address(String city, String country) {}  
public record Person(String name, int age, Address address) {}
```

```
public class NestedPatternExample {  
    public static void main(String[] args) {  
        Object obj = new Person("Bob", 40, new Address("New York", "USA"));  
  
        if (obj instanceof Person(String name, int age, Address(String city, String country))) {  
            System.out.println("Name: " + name + ", Age: " + age);  
            System.out.println("City: " + city + ", Country: " + country);  
        }  
    }  
}
```

```
}
```

Output:

Name: Bob, Age: 40

City: New York, Country: USA

Example: Record Patterns in Switch Statements

```
public class SwitchExample {  
  
    public static void main(String[] args) {  
  
        Object obj = new Person("Charlie", 25, new Address("London", "UK"));  
  
        switch (obj) {  
  
            case Person(String name, int age, Address(String city, String country))  
                -> System.out.println(name + " lives in " + city + ", " + country);  
  
            default -> System.out.println("Unknown object");  
  
        }  
  
    }  
}
```

Output:

Charlie lives in London, UK

Comparison of Java 21 vs. Older Versions

Feature	Java 21 (Record Patterns)	Java 20 and Earlier
Destructuring	Directly deconstruct record components	Requires explicit calls to accessor methods
Pattern Matching in Switch	Supports record patterns	Not supported
Nested Matching	Fully supported	Not supported
Boilerplate Code	Significantly reduced	More verbose

Pattern Matching for switch [JEP-441]

Pattern Matching for switch enhances the switch construct with pattern matching. This allows developers to perform refined data type checks, destructuring, and conditional logic in a more concise and declarative manner. Below are some of the key Enhancements:

- 1. **Type Patterns in switch:**
 - Enables switch to match on types directly without requiring casting.
- 2. **Guarded Patterns:**
 - Supports additional conditions (guards) for fine-grained control.
- 3. **Null Handling:**
 - Explicitly handles null in switch statements, improving safety.
- 4. **Exhaustiveness Checking:**
 - Ensures all possible cases are covered, improving reliability.
- 5. **Seamless Integration with Existing Features:**
 - Combines with pattern matching in instanceof and record patterns.

Example: Basic Pattern Matching with switch

Java 21 Implementation

```
public class SwitchPatternExample {  
  
    public static void main(String[] args) {  
  
        Object obj = "Hello, Java!";
```

```

switch (obj) {
    case String s -> System.out.println("It's a string: " + s);
    case Integer i -> System.out.println("It's an integer: " + i);
    default -> System.out.println("Unknown type");
}
}
}

```

Output:

It's a string: Hello, Java!

Example: Guarded Patterns

```

public class GuardedSwitchExample {
    public static void main(String[] args) {
        Object obj = 42;

        switch (obj) {
            case Integer i when i > 50 -> System.out.println("Large integer: " + i);
            case Integer i -> System.out.println("Small integer: " + i);
            case String s -> System.out.println("It's a string: " + s);
            default -> System.out.println("Unknown type");
        }
    }
}

```

Output:

Small integer: 42

Example: Null Handling

```

public class NullHandlingSwitch {
    public static void main(String[] args) {
        Object obj = null;

        switch (obj) {
            case null -> System.out.println("It's null");
            case String s -> System.out.println("It's a string: " + s);
            default -> System.out.println("Unknown type");
        }
    }
}

```

Output:

It's null

Comparison of Java 21 vs. Older Versions

Feature	Java 21 (Pattern Matching for switch)	Java 20 and Earlier
Type Matching	Directly in switch cases	Requires instanceof and casting
Guards (Additional Conditions)	Fully supported	Not supported
Null Handling	Explicitly supported	Requires separate if conditions
Conciseness	Reduces verbosity	More verbose

Unnamed Patterns and Variables (Preview) [JEP-443]

Unnamed Patterns and Variables introduced in Java 21 as a preview feature. It focuses on simplify code when certain parts of a pattern match are unnecessary. This enhancement allows us to ignore parts of a pattern or avoid declaring unused variables explicitly.

- 1. Unnamed Patterns (`_`):**
 - Use `_` to match a part of a pattern without binding it to a variable.
 - Useful for ignoring components we don't need in our logic.
- 2. Unnamed Variables (`_`):**
 - Use `_` to declare a variable without naming it.
 - Helps in avoiding warnings for unused variables in code.
- 3. Simplifies Record and Type Matching:**
 - Makes record and type pattern matching more concise and cleaner by ignoring unnecessary components.

Examples of Unnamed Patterns

Consider a Point record:

```
public record Point(int x, int y) {}
```

Java 21 Implementation:

```
public class UnnamedPatternExample {
    public static void main(String[] args) {
        Point point = new Point(10, 20);

        // Only match the `x` component and ignore `y`
        if (point instanceof Point(int x, _)) {
            System.out.println("X-coordinate: " + x);
        }
    }
}
```

Switching on Unnamed Patterns

```
public class SwitchWithUnnamedPatterns {
    public static void main(String[] args) {
        Object obj = new Point(15, 25);

        switch (obj) {
            case Point(int x, _) -> System.out.println("Point with X: " + x);
            case String _ -> System.out.println("It's a string");
            default -> System.out.println("Unknown type");
        }
    }
}
```

Output:

Point with X: 15

Examples of Unnamed Variables

Avoid Unused Variable Warnings

```
public class UnnamedVariableExample {  
  
    public static void main(String[] args) {  
  
        int[] numbers = {1, 2, 3, 4};  
  
        for (int _ : numbers) { // Unused variable `_`  
            System.out.println("Processing...");  
        }  
    }  
}
```

Output:

Processing...
Processing...
Processing...
Processing...

In Java 20 and earlier, we would need to name the variable explicitly, even if unused, which could lead to warnings or confusion.

Comparison of Java 21 vs. Earlier Versions

Feature	Java 21 (Unnamed Patterns/Variables)	Java 20 and Earlier
Ignoring Components	Supported using _	Not possible; all components must be named
Unused Variables	Use _ to avoid warnings	Explicit naming required
Conciseness	Cleaner and more expressive patterns	Verbose and sometimes cluttered

Use Cases

- 1. **Reducing Boilerplate in Pattern Matching:**
 - Ignore fields or variables that are not needed for logic.
- 2. **Improved Code Readability:**
 - Makes intent explicit by visually distinguishing irrelevant parts.
- 3. **Enhanced Compatibility with Future Features:**
 - Prepares for more robust pattern-matching capabilities in Java.

Note: Use the --enable-preview flag during compilation and execution of any preview feature:

```
javac --enable-preview --release 21 Example.java  
  
java --enable-preview Example
```

Unnamed Classes and Instance main Methods (Preview) [JEP-445]

Unnamed Classes and Instance main Methods, introduced as a preview feature in Java 21. It simplifies the development of small programs by allowing unnamed classes and instance-based main methods. This enhancement is part of Java's effort to make it easier to write, execute, and explore Java programs without boilerplate code.

- 1. **Unnamed Classes:**
 - Allows us to write Java code in an unnamed class directly, without explicitly declaring a class name.
 - Focuses on quick prototyping and scripting use cases.
- 2. **Instance main Methods:**
 - The main method can now be written as an **instance method**, removing the requirement for it to be static.
 - Simplifies small programs by enabling instance-based logic directly in the entry point.

Examples: Unnamed Classes

Java 21 Implementation:

```
// Save as Example.java
```

```
void main() {  
    System.out.println("Hello, Java 21!");  
}
```

Execution:

```
>java --enable-preview --source 21 Example.java
```

Output:

Hello, Java 21!

In Java 20 and earlier, we would have to define a named class explicitly as below code snippet:

```
public class Example {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

Example: Instance main Methods

Java 21 Implementation:

```
public class InstanceMainExample {  
    public void main() {  
        System.out.println("This is an instance-based main method.");  
    }  
}
```

Execution:

```
>java --enable-preview --source 21 InstanceMainExample.java
```

Output:

This is an instance-based main method.

In Java 20 and earlier, main had to be a static method:

```
public class StaticMainExample {  
    public static void main(String[] args) {  
        System.out.println("This is a static main method.");  
    }  
}
```

Comparison of Java 21 vs. Earlier Versions

Feature	Java 21 (Unnamed Classes & Instance main)	Java 20 and Earlier
Unnamed Classes	Supported	Not possible
Instance main Methods	Supported	Only static main allowed
Ease of Use	Simplifies small programs	More boilerplate for similar tasks
Prototyping	Ideal for quick tests and prototyping	Requires full class and method definitions

Scoped Values (Preview) [JEP-446]

Scoped Values, introduced as a preview feature in Java 21. It provides a new mechanism for sharing immutable data across methods and threads in a safe and predictable way. They are designed as a structured alternative to thread-local variables, which can sometimes lead to complexities and errors in concurrent programming.

Key Features

1. **Scoped Values:**
 - Represent immutable data that can be accessed within a well-defined scope.
 - Safer and more predictable than ThreadLocal variables.
2. **Concurrency-Friendly:**
 - Designed for use in multi-threaded environments.
 - Eliminates many of the pitfalls associated with ThreadLocal.
3. **Cleaner API:**
 - Encourages clearer and more modular code by keeping shared state within explicit scopes.

Use Case Comparison

ThreadLocal in Older Versions

Before Java 21, developers often used ThreadLocal to store data specific to a thread, like this:

```
public class ThreadLocalExample {  
  
    private static final ThreadLocal<String> context = ThreadLocal.withInitial(() -> "Default");  
  
    public static void main(String[] args) {  
  
        context.set("Main Thread Context");  
  
        new Thread(() -> System.out.println("Child Thread: " + context.get())).start();  
  
        System.out.println("Main Thread: " + context.get());  
  
    }  
}
```

Although this approach is functional, but has limitations:

- **Memory Leaks:** ThreadLocals can hold references longer than intended.
- **Complex Debugging:** Hard to trace values through thread transitions.

Scoped Values in Java 21

Scoped values provide a structured alternative:

```
import java.lang.ScopedValue;  
  
public class ScopedValueExample {  
  
    private static final ScopedValue<String> CONTEXT = ScopedValue.newInstance();  
  
    public static void main(String[] args) {  
  
        ScopedValue.where(CONTEXT, "Main Thread Context").run() -> {  
            new Thread(() -> {  
                ScopedValue.where(CONTEXT, "Child Thread Context").run() -> {  
                    System.out.println("Child Thread: " + CONTEXT.get());  
                });  
            }).start();  
  
            System.out.println("Main Thread: " + CONTEXT.get());  
        });  
    }  
}
```

Output:

Main Thread: Main Thread Context

Child Thread: Child Thread Context

Comparison of Scoped Values vs. ThreadLocal

Feature	Scoped Values (Java 21)	ThreadLocal (Earlier Versions)
State Sharing	Immutable and scope-bound	Mutable and thread-bound
Memory Management	No risk of leaks	Can cause memory leaks
Concurrency Support	Designed for multi-threading	Limited; can be error-prone
Ease of Use	Cleaner API and usage	Requires manual management

Advantages of Scoped Values

- 1. **Safer State Management:**
 - Scoped values eliminate unintended side effects by making state immutable and scope-bound.
- 2. **Cleaner Code:**
 - Reduces boilerplate and clarifies intent by explicitly defining the scope of shared state.
- 3. **Better Debugging:**
 - Scoped values are easier to trace and debug compared to ThreadLocal.
- 4. **Optimized for Concurrent Use:**
 - Scoped values are designed for safe usage in multi-threaded environments, making them ideal for modern Java applications.

Use Cases

- 1. **Context Propagation:**
 - Use scoped values for passing contextual information like user identity or transaction details across threads in web servers or distributed systems.
- 2. **Simplified Frameworks:**
 - Frameworks can use scoped values to manage per-request state without relying on ThreadLocal.
- 3. **Enhanced Security:**
 - By limiting the scope of shared state, scoped values reduce the risk of data leaks or unauthorized access.